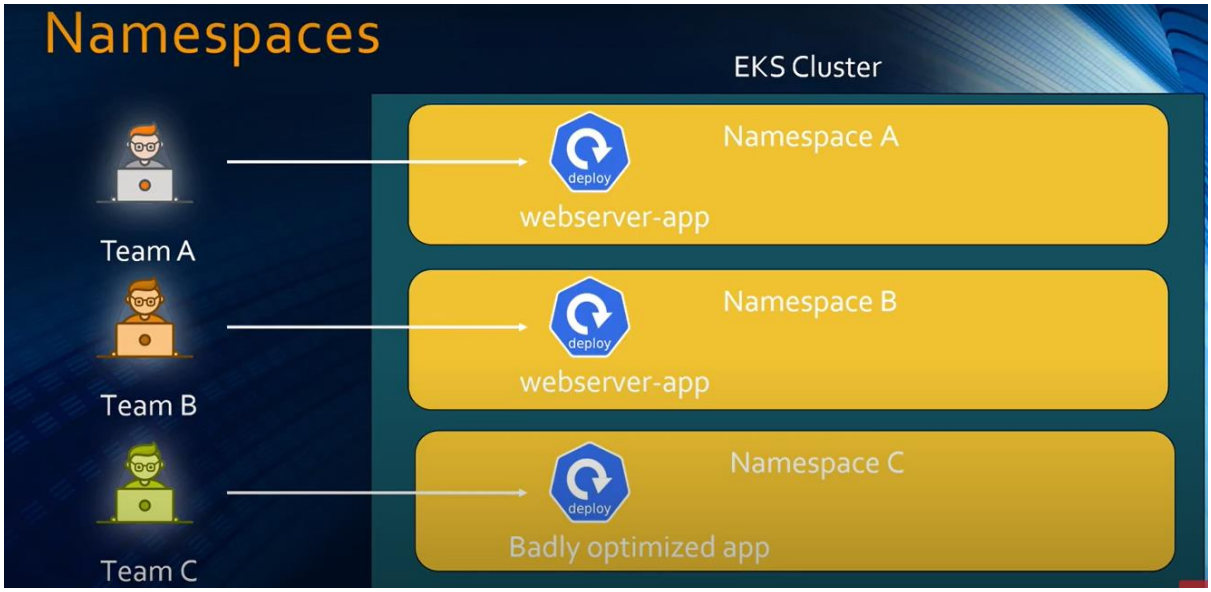
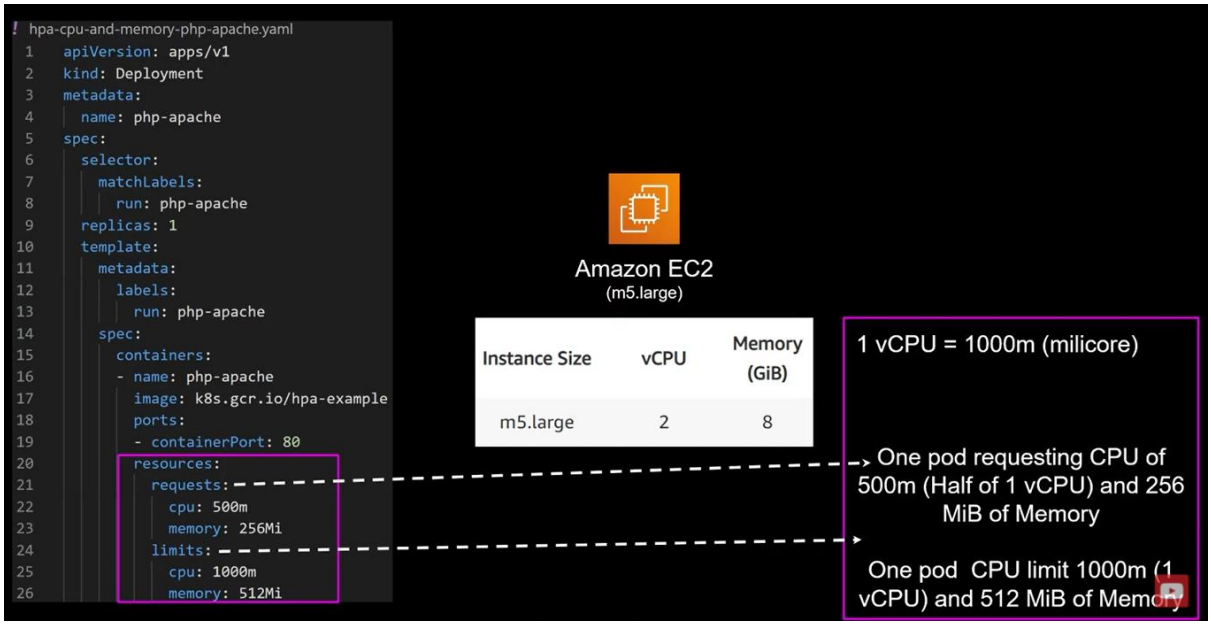
1. **Namespaces**:
    - Multiple virtual cluster boundary within one physical cluster.
    - Provides scope for naming.
    - Physically allocation of memory utilization
    - Divide cluster resources to namespaces
    - Useful for namespace specific accesses



2. Resource Requests and Limits

- Set appropriate requests and limits for container
- Set resources for Namespaces using ResourceQuotas
- When CPU reaches limit, it'll throttle (observe slowness of program)
- When Memory reaches limit, Pod will be evicted
- Proper requests and limits save cost as well.

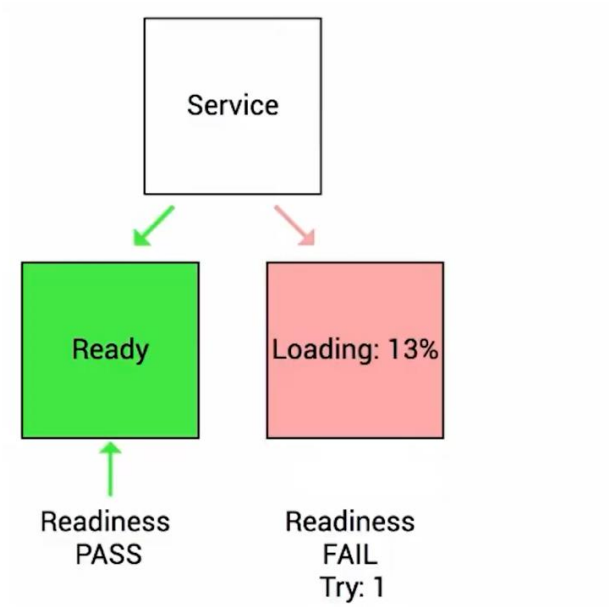


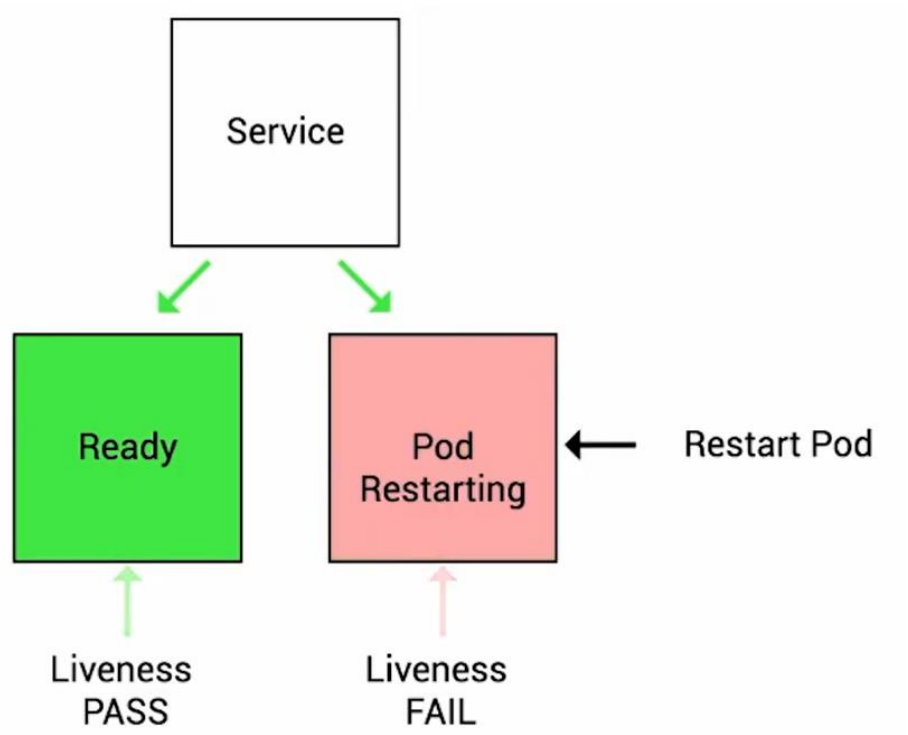
How do you cost/performance optimize K8s app?

Detect CPU/Memory waste=> Utilize Metrics server

3. Use Readiness and Liveness Probe

- Your service will not work until your application up and running if your applications take 2-3 minute for warm-up?
- By default, K8s starts sending traffic to container as soon as process inside container starts
- By using Readiness probe K8s waits until application fully started, service will not send the traffic.
- Application stuck e.g., your cpu reaches limit and your application throttling very badly, K8s things everything is fine and continually sending traffic to this pod





4. Secure K8s
    - K8s Security
        i. Application Security
            1. Pod, Namespace, Node
            2. RBAC, IRSA
        ii. DevSecOps->DevOps + Security-> Security of the container DevOps lifecycle
            1. Authorization
            2. Scan Repository

3. Scan running container

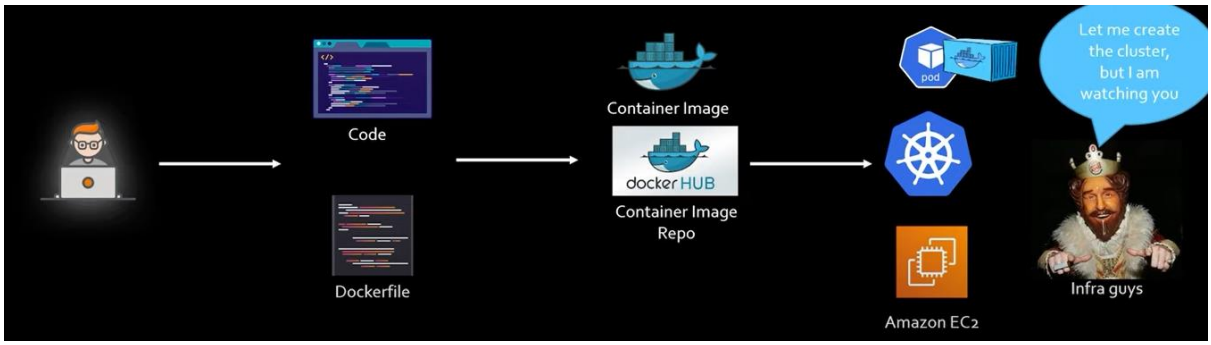
iii. Security Compliance

1. FedRAMP, HIPPA, SOC etc

5. Ops

- Detective Controls
  - i. Collect and Analyse audit logs
  - ii. Alarm on certain behaviour
- Understand K8s termination lifecycle
  - i. Drain nodes
  - ii. Use rolling Update
  - iii. App handle gracefully termination
- Incident response
  - i. Identify and isolation
  - ii. Run load and penetration testing

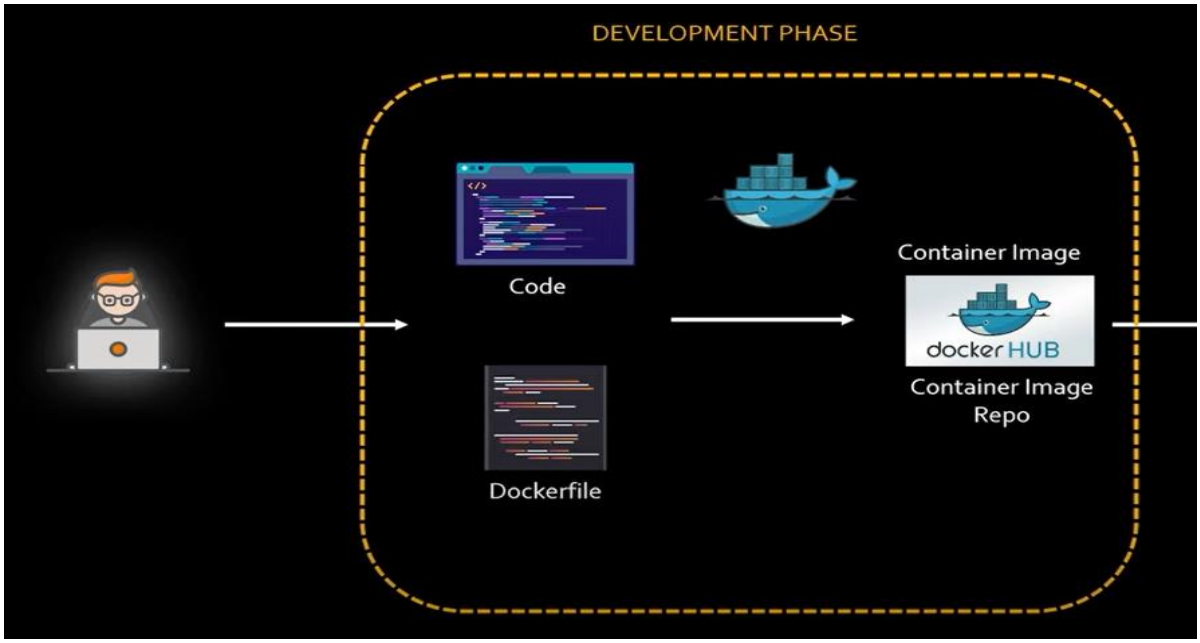
K8s Security best practices?



Development phase:

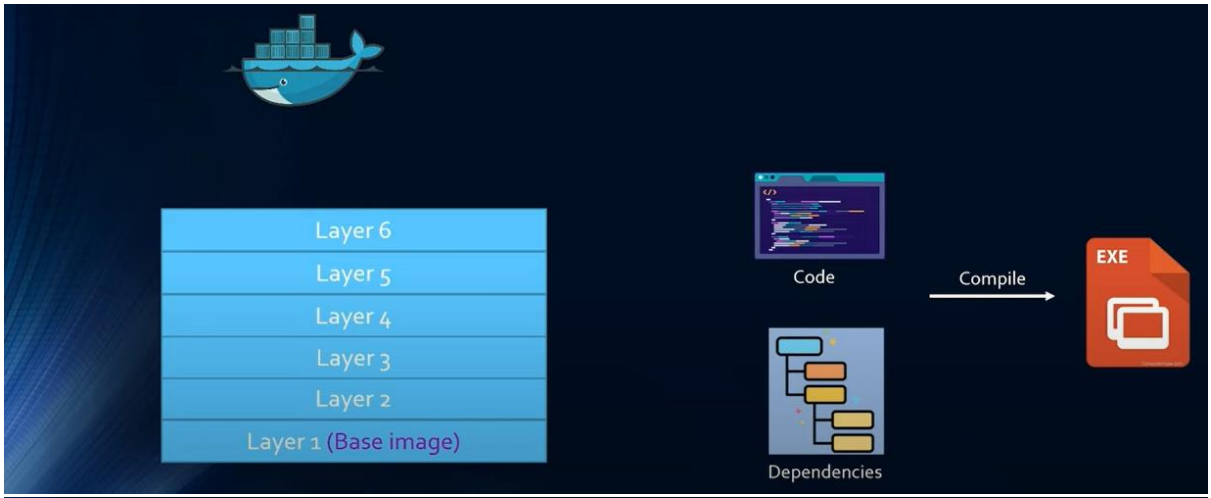
Code

Docker Image

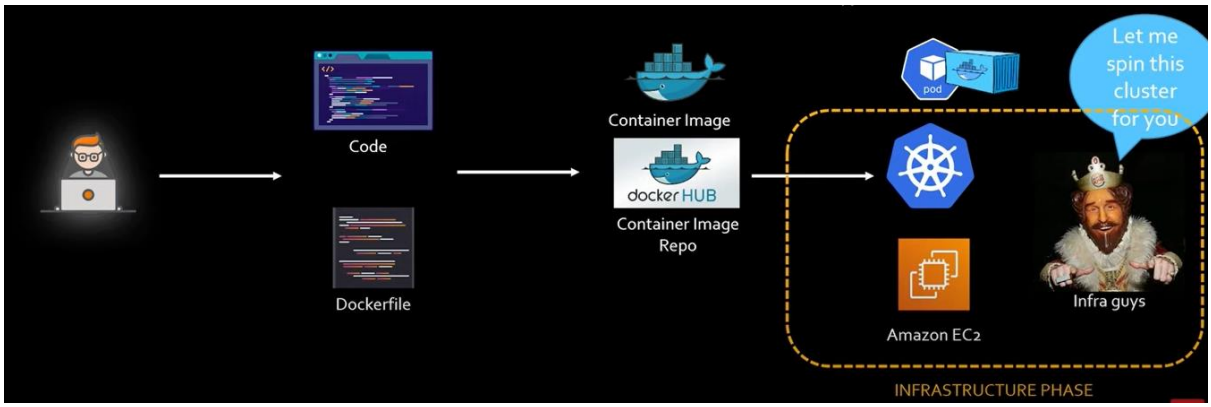


Development Phase Security Best Practices

- Create minimal images
- Reducing package, reduces attack surface
- Use multi-stage builds to reduce size
- Reducing the number of layers
- Run static scan for vulnerabilities
- Keep some scanning tools names handy (Clair, TwistLock, AquaSec)
- Use private repository



Infrastructure phase:



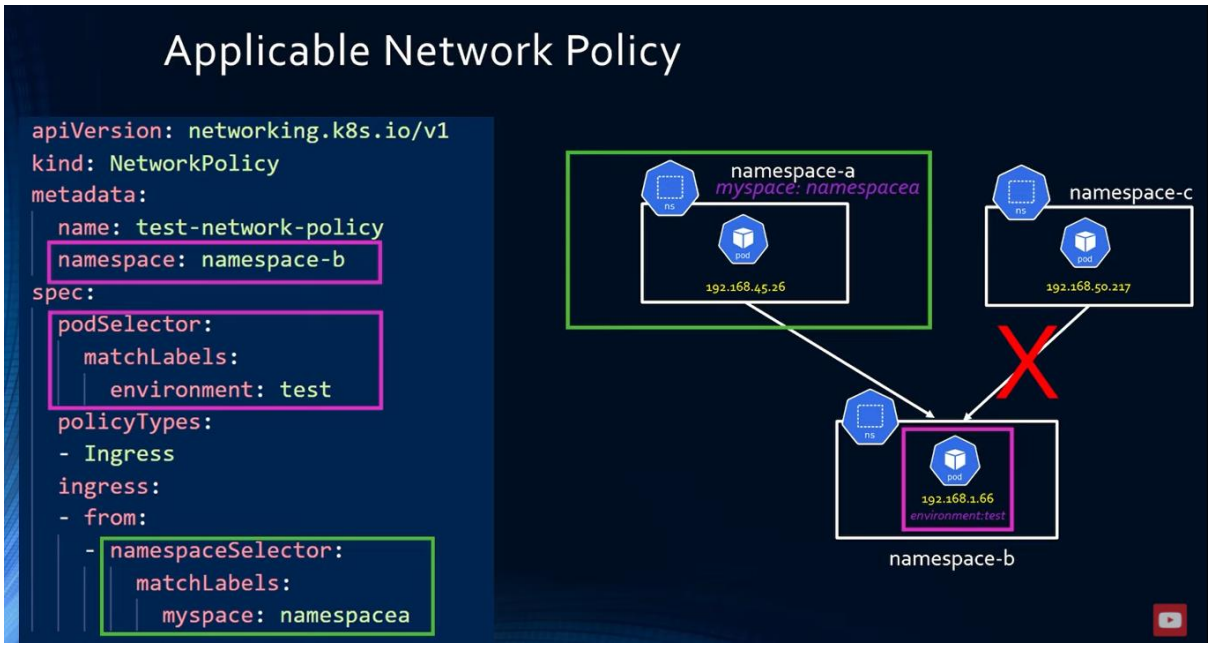
Infrastructure Phase Security Best Practices

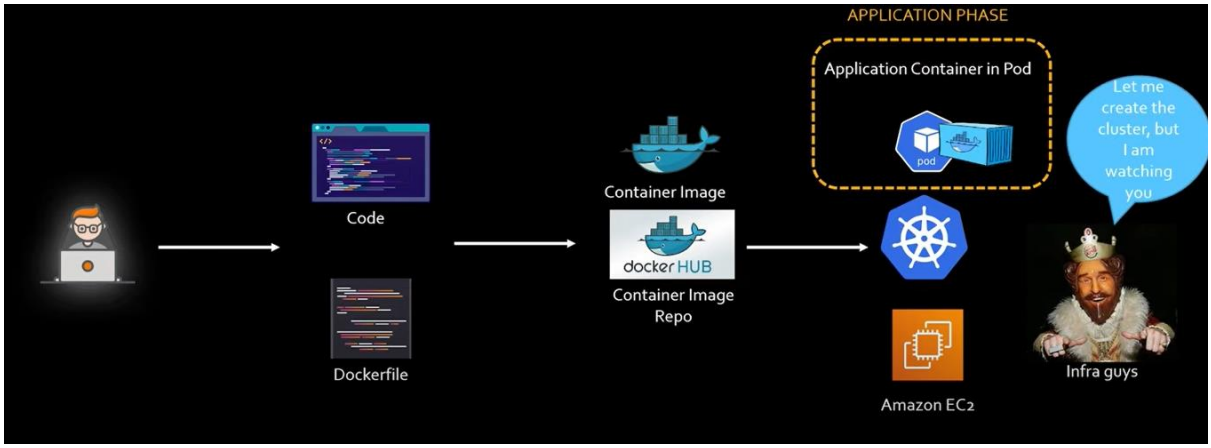
- Use hardened AMIs
  - Reducing packages, reduces attack surface
- Run latest K8s version
  - Lots of security patches
- Run kube-bench for CIS benchmarking periodically (Vulnerability)
  - AWS X-Ray

Application phase:

Application Phase Security Best Practices

- Use namespaces to divide the cluster
  - Separate resource quota and access for each namespace
  - By default, all pods can talk to each other
- Use network policy to control pod traffic
  - Control traffic by IP, label, or namespace
- Implements RBAC
  - Specify separate roles for separate groups (admin, developer)
- Do not allow privileged escalation
  - Prepare for privileged Vs root
- Use OPA (Open Policy Agent) to enforce restrictions
  - Images are approved repo, namespaces with label with point of contact etc...
  - Using Gatekeeper





Detection phase:

Detection Phase Security Best Practices

- Run dynamic scan on running container
  - Keep some scanning tools names handy (TwistLock, AquaSec, Snyk)
- Enable audit logs
  - Create Alarm